Computer Engineering Dept.
Faculty of Engineering
Cairo University

Advanced Databases

Semantic Search Engine Using Vectorized Database - Phase 2

Team 13

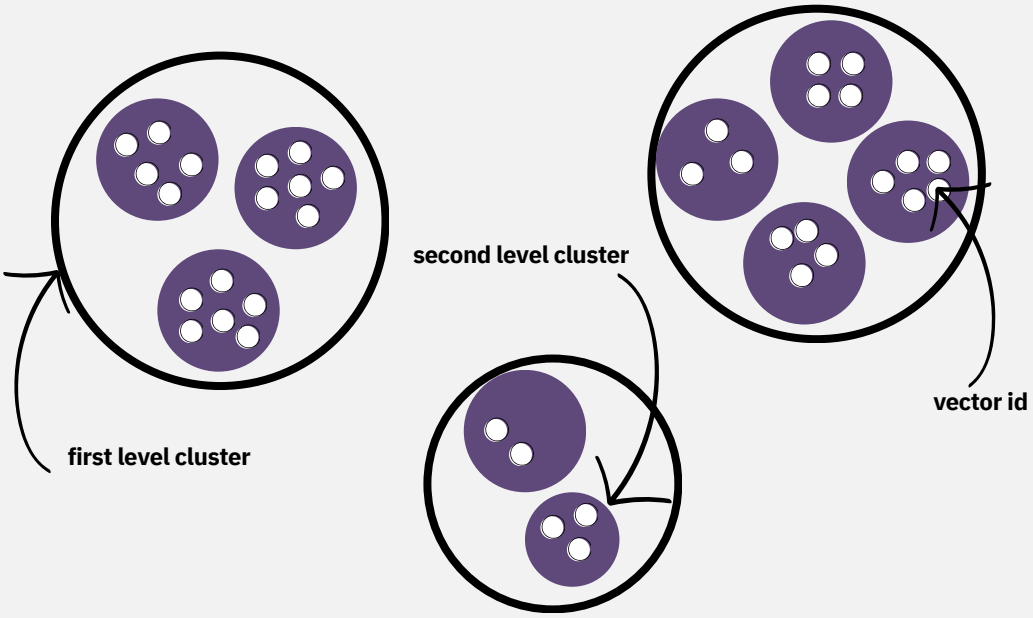
NAME	BN	SECTION
Rawan Mostafa Mahmoud Abdel Samad	**18**	**1**
Fatma Ebrahim Sobhy	**1**	**2**
Menna Mohamed Abdelbaset	**26**	**2**
Sara Bisheer Fikry	**19**	**1**

Trials:

1. **One-level IVF:**
    a. We've tried 1-level IVF at first, sample score we got was (score: -74.7,time: 2.7307076692581176) with normal list sorting
    b. Then we tried to use heap sorting to reduce time and RAM
2. **One-level IVF with PQ:**
    a. Adding PQ for small 1k db with index file size 36kB, score -229.8, time 0.88, but we were mistakenly understanding that we should store the vectors themselves in the index file when we got these numbers, this is why the index file size is larger than the db rows
    b. We've noticed that PQ takes very long time to build the index files, so we couldn't try much things with it
    c. We've decided to try 2-level IVF and see if we got better results

Two-level IVF - Final Decision

- ## 2-level IVF visualization



- ## Index Building Steps:
  a. Use MiniBatchKMeans to cluster all database rows into the 1st level clusters, create a map `cluster_mapping` that has the key as the centroid and value is its corresponding vector ids
  b. Dump the first level centroids into a file --> **centroids.dat**
  c. Loop on each centroid in the first level, Apply MiniBatchKMeans to cluster the vectors inside each cluster
  d. Dump a file containing a dictionary having the keys as the 2nd level centroids and the value as vector ids falling into this cluster -->i.e. **clusters/0.dat**

Now our index file consists of two parts:

1. First level centroids file
2. Cluster mapping of the second level as a directory containing a file for each 1st level cluster, having all the 2nd level clusters inside of it and the vector ids corresponding to each 2nd level cluster

- **Data Structures Used to build the index:**
  - **centroids:** list of 1st level cluster centroids
  - **cluster_mapping:** dictionary of {”1st level cluster centroids”:”vector ids”}
    - Note: this DS is not stored as an index file on the disk, it’s only used as a variable
  - **cluster_vector_ids:** list having the vector ids that belong to each first level cluster (used to get these vectors from database all at once)
  - **centroids_2:** list of 2nd level cluster centroids
  - **cluster_mapping_2:** dictionary of {”2nd level cluster centroids”:”vector ids”}
    - Note: this DS is the one that’s actually stored in the index file

Records Number	level one batch size	level two batch size	n_cluster1	max_iter
1M	10	10	4000	200
10M	10	10	7000	200
15M	750	10	10000	1000
20M	1000	10	13000	1000

Note : n_clusters2 is decided as the min number of vectors inside all 2nd level clusters

- **Retrieve Steps:**
  a. Load first level centroids file **(centroids.dat)**
  b. Calculate the distance between all 1st level centroids and get nearest **nprobe_1** clusters
  c. Loop on each nearest cluster centroid, read its corresponding index file i.e. **clusters/0.dat**
  d. Calculate the distance between all 2nd level centroids and get nearest **nprobe_2** clusters
  e. Loop on each nearest cluster centroid, read its vectors from db on batches, max batch size is calculated as it's the available RAM **(RAM constraint) // (vector size * scaling factor)**, the scaling factor is used to give a space for other variables stored in the memory at the same time, meaning that vectors can't take up the whole available RAM, it should leave space for other variables.
  f. Calculate distance between the query vector and all vectors in this batch, sort them using heap sort and take **top_k** vectors

Records Number	n_probe1	n-probe2	scaling factor
1M	90	45	500
10M	80	40	500
15M	140	70	1000
20M	150	75	4000

- # Index file storage Trials:

- **/clusters_1st**
  - **/0.dat -->** each fist level cluster file containing 2nd level cluster centroids falling inside of it
- /clusters_2nd
  - **0.dat -->** each second level cluster file containing vector ids falling inside of it

But this was very hard to manage to open the directories and files and keep track of corresponding centroids to cluster ids so we decided to store the centroids itself

- {”centroid_1st”:
  {
  “centroid_2nd”: [vector ids],
  “centroid_2nd”:[vector_ids],
  },
  ”centroid_1st”:
  {
  “centroid_2nd”: [vector ids],
  “centroid_2nd”:[vector_ids],
  },
  }

but this made the RAM very large because it was loading all centroids at once

- **current approach:**
  - list [1st level centroids] --> centroids.dat
  - {”2nd_level_centroids”:[vector_ids]} -->clusters/i.dat
  - This approach helped us reduce the RAM usage as we only read the nearest n_probes from the second dictionary, also was easier to manage as we stored the centroids itself

- **Trade-offs**
  - **Time x RAM**
    - **Scaling factor:** as the scaling factor increases, RAM usage decreases but the time increases
  - **Time x Accuracy**
    - **n_probe:** as the n_probe increases, the Accuracy increases, but time also increases
    - **n_clusters:** for constant n_probes, as n_clusters increases, time decreases but accuracy also decreases

- **Number of Calculations**
  - **Index Building - 1st level**
    - For each batch: Batch_dot_products = n_clusters * batch_size = 13K * 1K = 13M
    - For each iteration: #batches * Batch_dot_products= (20M / 1K) * 13M = 260G
    - For all iterations: 1K* 260G = 260,000G
    - For predict: 20M×13K=260G
    - Total till now = 260260G

- **Number of Calculations**
  - **Index Building - 2nd level**
  - On average each cluster will contain = ceil(20M/13K) =1539 row data
  - Assume n_clusters_2 = 1538
    - For each batch: Batch_dot_products = n_clusters * batch_size = n_clusters_2 * 10 = 15380
    - For each iteration: #batches * Batch_dot_products= (1539 / 10) * 15380 = 2366982
    - For all iterations: 1,000 * 2366982 = 2366982K
    - For predict: 20M × 1538 = 307,600M
    - Total = 307,600M + 2,366,982K +260,260G

  - **Retrieve**
    - Total = n_clusters_1 + nprobe_1 * n_clusters_2 + nprobe_2 * n_clusters_2 = 13,000 + (150 * 1538) + (75 * 1538) = 359,050

Results

- **seed 10**
  - 1M score 0.0 time 1.68 RAM 9.61 MB
  - 10M score 0.0 time 5.60 RAM 23.23 MB
  - 15M score -5.333333333333333 time 5.63 RAM 12.68 MB
  - 20M score 0.0 time 7.38 RAM 7.74 MB

- **seed 29**
  - 1M score 0.0 time 1.49 RAM 8.50 MB
  - 10M score 0.0 time 4.20 RAM 22.25 MB
  - 15M score 0.0 time 5.59 RAM 11.32 MB
  - 20M score 0.0 time 6.65 RAM 3.04 MB

- **seed 256**
  - 1M score 0.0 time 1.63 RAM 8.60 MB
  - 10M score -20.333333333333332 time 4.98 RAM 22.36 MB
  - 15M score 0.0 time 6.05 RAM 11.32 MB
  - 20M score 0.0 time 6.82 RAM 3.55 MB

- **seed 3**
  - 1M score 0.0 time 1.68 RAM 8.50 MB
  - 10M score 0.0 time 4.97 RAM 22.36 MB
  - 15M score 0.0 time 6.37 RAM 11.34 MB
  - 20M score 0.0 time 6.71 RAM 3.76 MB

- **seed 12**
  - 1M score 0.0 time 1.97 RAM 8.50 MB
  - 10M score 0.0 time 4.90 RAM 22.16 MB
  - 15M score 0.0 time 6.25 RAM 11.11 MB
  - 20M score -5.333333333333333 time 6.68 RAM 4.01 MB